

VNA Reference Plane Shifting via SOL Calibration

Sipan Hovsepian

1. PROBLEM STATEMENT

In an ideal Vector Network Analyzer (VNA) measurement, the Device Under Test (DUT) is connected directly to the VNA ports to minimize signal attenuation and phase errors. However, physical constraints often make direct connection impossible.

As illustrated in Figure 1, DUT in our setup is connected through a PSB board with embedded SP4T switch and traces, which alter the original reading. The goal, illustrated in Figure 2 is to shift reference plane to the end of SMA connector embedded on the PCB, where DUTs will be connected.

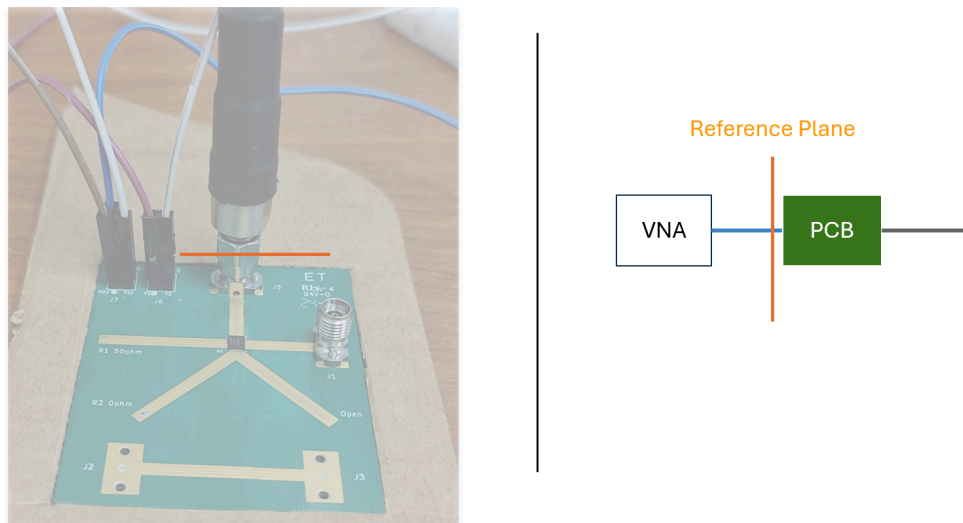


Figure 1: Default reference plane

Published May 05, 2026

Acknowledgements

This research made use of **scikit-rf** (www.scikit-rf.org), an open-source Python package for RF and Microwave applications.

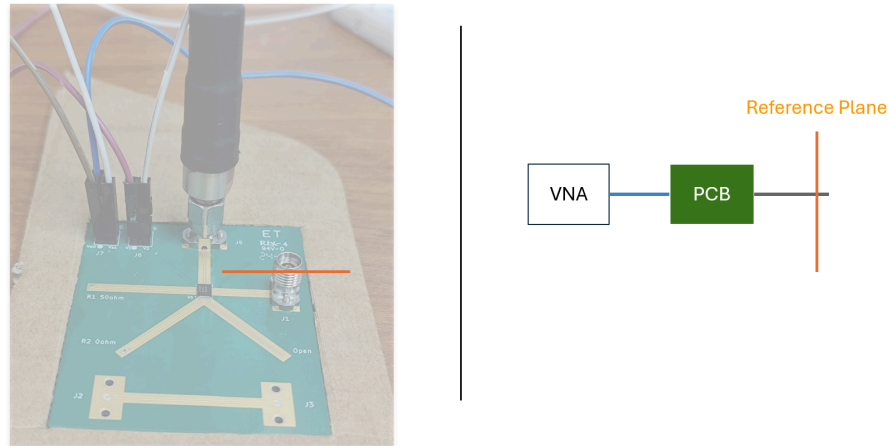


Figure 2: Desired Reference plane

1.a. Objective: Reference Plane Shifting

To obtain an accurate characterization of the DUT alone, we must mathematically “shift” the measurement reference plane from the VNA port to the actual input terminals of the DUT. This process allows us to:

- **Remove Fixture Effects:** De-embed the parasitic inductance and capacitance of the PCB interface.
- **Isolate DUT Response:** Ensure the measured S-parameters reflect only the component under study, not the delivery network.

1.b. Experiment Parameters

This section defines the experimental setup and the file naming protocol used to categorize the 1-port (S11) data files generated by the VNA for de-embedding analysis.

1.c. Data Acquisition & VNA Setup

The following parameters were consistent across all measurements:

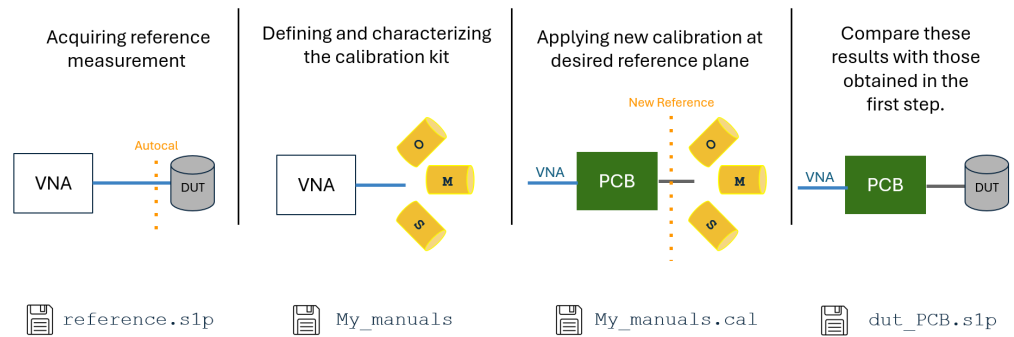
Parameter	Value
Measurement Type	S11 (1-Port Reflection)
Start Frequency	100 MHz
Stop Frequency	18 GHz
Number of Points	1601
IF Bandwidth	1 kHz

2. APPROACH 1: FIRMWARE BASED APPROACH

We can utilize Rohde and Swartz ZNL20’s firmware to accomplish our goal.

2.a. Methodology

Methodology is illustrated in [Figure 3](#)



2.b. Video tutorial



2.c. Results

Results are illustrated in [Figure 4](#)



3. APPROACH 2: SCIKIT-RF IMPLEMENTATION

3.a. Methodology

The following implementation utilizes the `scikit-rf` library to perform an single port Short-Open-Load (SOL) calibration. By comparing the measured responses of the standards through the PCB fixture against their ideal models, we calculate the error terms necessary to shift the reference plane and isolate the DUT response from the surrounding parasitics.

3.b. Calibration Standards: Reference Plane Definition

To accurately de-embed the fixture, two distinct sets of Short-Open-Load (SOL) measurements were acquired. These images illustrate the physical locations used to define the “Ideal” and “Measured” networks.

- **Ideal Standards** were measured by connecting the SOL standards directly to the VNA’s wire end.

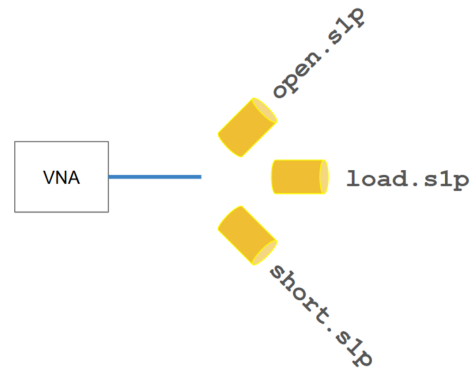


Figure 5: Ideal calibration reference plane, defined at the end of the VNA's wire.

- **Measured Standards** were acquired by placing each standard at the desired measurement reference plane, for instance, at the antenna port on the PCB fixture.

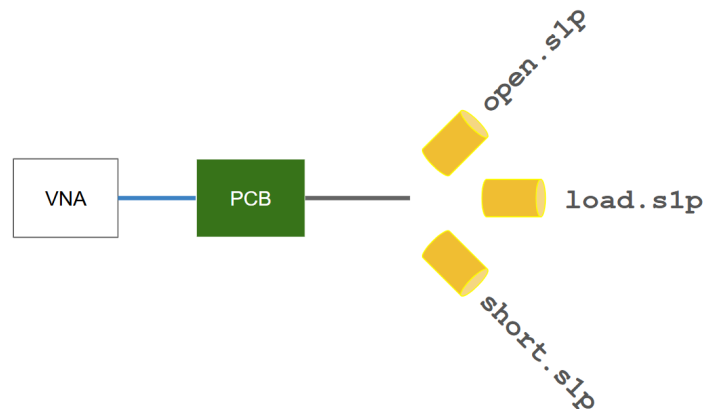


Figure 6: Measured calibration reference plane, defined at the antenna port on the PCB fixture.

3.c. Calibration Implementation

```
import skrf as rf
from skrf.calibration import OnePort
import matplotlib.pyplot as plt

rf.stylely()

my_ideals = rf.io.load_all_touchstones('ideals_calib_OFF/')
my_measured = rf.io.load_all_touchstones('PCB_wire/measured/')

cal = OnePort(
    ideals = [my_ideals[k] for k in ['short', 'open', 'load']],
    measured = [my_measured[k] for k in ['short', 'open', 'load']],
)
```

```
cal.run()

dut_original = rf.Network('dut_PCB_wire.slp')
dut_caled = cal.apply_cal(dut_original)
dut_caled.name = dut_original.name + ' corrected'

ground_truth = rf.Network('dut.slp')

plt.figure()
dut_original.plot_s_db()
dut_caled.plot_s_db()
ground_truth.plot_s_db()
```

3.d. Results

This section presents the corrected S-parameter results following the `scikit-rf` de-embedding process. Each figure compares the original raw measurement against the de-embedded (corrected) data, using the direct DUT measurement as the ground truth.

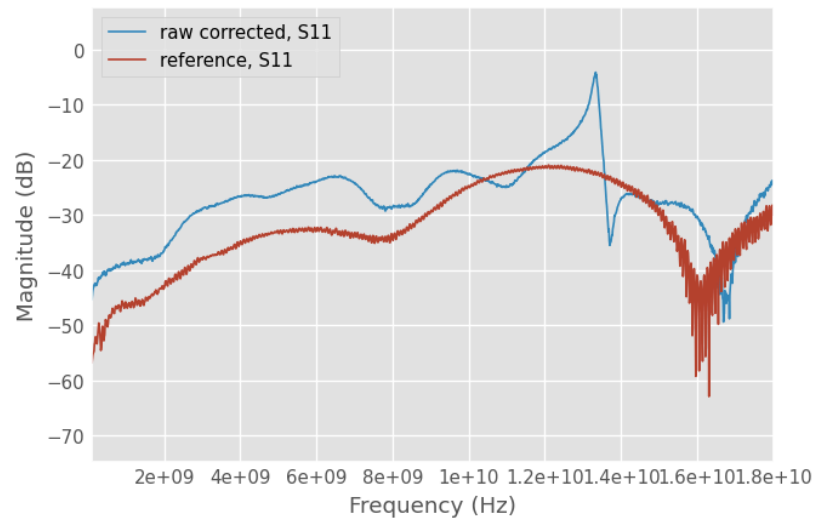


Figure 7: Comparison of raw wire measurement, de-embedded result, and ground truth.

3.e. Citation

Arsenovic et al., “scikit-rf: An Open Source Python Package for Microwave Network Creation, Analysis, and Calibration [Speaker’s Corner],” in *IEEE Microwave Magazine*, vol. 23, no. 1, pp. 98-105, Jan. 2022, doi: 10.1109/MMM.2021.3117139.

4. APPROACH 3: MATHEMATICAL IMPLEMENTATION

Although the VNA firmware and scikit-rf provide satisfactory results, both wrap the underlying mathematical formulations into high-level functions. For users interested in the derivation, this section provides the theory and a direct Python implementation.¹

The one-port error model relates the **actual** reflection coefficient Γ_a to the **measured** reflection coefficient Γ_m through three error terms: directivity E_D , source match E_S , and reflection tracking E_R :

$$\Gamma_m = E_D + \frac{E_R \Gamma_a}{1 - E_S \Gamma_a} \quad (1)$$

Applying three known standards (Open, Short, Load) yields a 3×3 linear system that is solved for the error terms at each frequency point. Once E_D , E_S , and E_R are known, the corrected DUT reflection coefficient is recovered as:

$$\Gamma_{DUT} = \frac{\Gamma_m - E_D}{E_R + E_S (\Gamma_m - E_D)} \quad (2)$$

The **actual** standards Γ_a are defined at the VNA cable end:

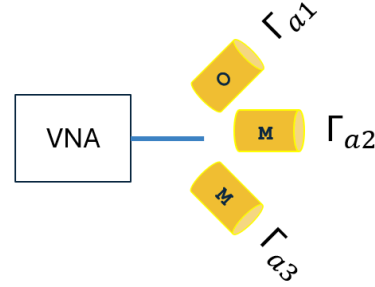


Figure 8: Actual standard definitions at the VNA cable reference plane.

The **measured** standards Γ_m are acquired at the PCB fixture port:

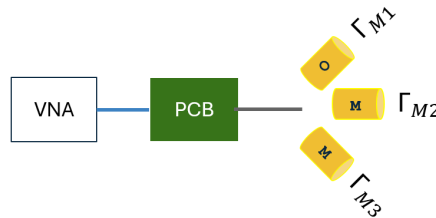


Figure 9: Measured standards at the PCB fixture reference plane.

¹"One-Port VNA Calibration: A Look Under the Hood", *Microwaves & RF*, 2014.

```

import skrf as rf
import matplotlib.pyplot as plt
import numpy as np
import math

numpoints = 1601

# --- Load ideal (actual) standards ---
openGamma = rf.Network('ideals/open.slp')
shortGamma = rf.Network('ideals/short.slp')
loadGamma = rf.Network('ideals/load.slp')

# --- Load measured standards ---
m_openGamma = rf.Network('measured/open.slp')
m_shortGamma = rf.Network('measured/short.slp')
m_loadGamma = rf.Network('measured/load.slp')

# --- Extract complex S11 arrays ---
def extract_s11(net):
    return net.s[:, 0, 0] # shape: (numpoints,)

Gamma_a = np.stack([extract_s11(openGamma),
                    extract_s11(shortGamma),
                    extract_s11(loadGamma)], axis=1) # (N, 3)

Gamma_m = np.stack([extract_s11(m_openGamma),
                    extract_s11(m_shortGamma),
                    extract_s11(m_loadGamma)], axis=1) # (N, 3)

# --- Solve for error terms at each frequency point ---
# System: C * [R, D, S]^T = V
# C[k,:] = [Gamma_a_k, 1, Gamma_a_k * Gamma_m_k]
# V[k] = Gamma_m_k

D = np.zeros(numpoints, dtype=complex) # Directivity (E_D)
S = np.zeros(numpoints, dtype=complex) # Source match (E_S)
R = np.zeros(numpoints, dtype=complex) # Refl. tracking (E_R)

for i in range(numpoints):
    ga = Gamma_a[i] # [open, short, load] actual
    gm = Gamma_m[i] # [open, short, load] measured

    C = np.column_stack([ga, np.ones(3), ga * gm]) # (3, 3)
    V = gm.reshape(3, 1)

    # Least-squares solve: E = (C^H C)^{-1} C^H V
    CH = C.conj().T
    E = np.linalg.solve(CH @ C, CH @ V).ravel()

    R[i] = E[0] + E[1] * E[2] # E_R = a + E_D * E_S
    D[i] = E[1] # E_D
    S[i] = E[2] # E_S

```



```

# --- Apply correction to DUT ---
dut      = rf.Network('raw.s1p')
dut_meas = dut.s[:, 0, 0]

dut_corr  = (dut_meas - D) / (R + S * (dut_meas - D))
dut_corr_db = 20 * np.log10(np.abs(dut_corr))

# --- Plot ---
plt.figure()
plt.plot(dut.f / 1e9, 20 * np.log10(np.abs(dut_meas)), label='Raw')
plt.plot(dut.f / 1e9, dut_corr_db, label='Corrected')
plt.xlabel('Frequency (GHz)')
plt.ylabel('S11 (dB)')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

```

4.a. Results

The following plot displays the raw and corrected measurements.

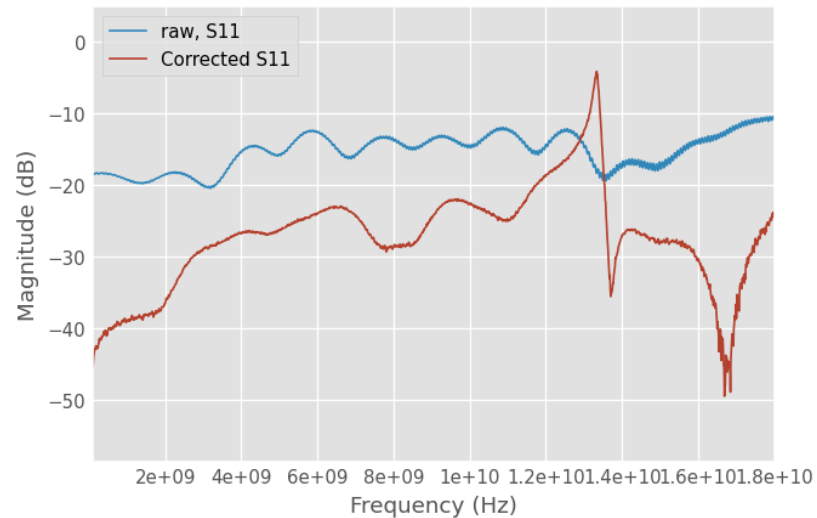


Figure 10: Comparison of raw wire measurement, de-embedded result, and ground truth.